

C# & ADO.NET

Module 3 — Programming Exercises

30 Complete Exercises with Sample Output

Table of Contents

#	Topic
1	Set Up the Development Environment
2	Explore Value vs Reference Types
3	Use Primary Constructors in C# 12
4	Demonstrate Type Inference with var and new()
5	Perform Conditional Logic for Grade Calculation
6	Loop Through an Array with Different Loop Types
7	Implement Method Overloading
8	Use ref, out, and in Parameters
9	Use Local Functions
10	Demonstrate OOP Basics with Constructors
11	Demonstrate Access Modifiers
12	Use Auto-Properties and Backing Fields
13	Create and Use Records with init Properties
14	Demonstrate Inheritance and Method Overriding
15	Differentiate Abstract Classes and Interfaces
16	Handle Null References Safely
17	Use Null-Conditional Chaining in a Contact App
18	Use the required Modifier in C# 12
19	Work with Lists and Dictionaries
20	Use LINQ for Filtering and Projection
21	Use Pattern Matching with is and switch
22	Create and Deconstruct Tuples
23	Simulate Async File Upload with Exception Handling
24	Serialize and Deserialize JSON Files
25	Use FileStream and MemoryStream
26	Demonstrate Race Conditions with Multi-threading
27	Simulate and Resolve a Deadlock
28	Log with System.Diagnostics.Trace
29	Sanitize Input and Prevent XSS
30	Perform CRUD Operations using ADO.NET

Exercise 01: Set Up the Development Environment

■ Source Code

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Hello, World!");
        Console.WriteLine("Environment setup successful.");
        Console.WriteLine($" .NET Version: {Environment.Version}");
    }
}
```

■ Sample Output

```
Hello, World!
Environment setup successful.
.NET Version: 8.0.0
```

Exercise 02: Explore Value vs Reference Types

■ Source Code

```
using System;

class MyData { public int Value { get; set; } }

class Program
{
    static void ModifyValue(int x)          { x = 100; }
    static void ModifyReference(MyData obj) { obj.Value = 100; }

    static void Main()
    {
        int a = 10;
        Console.WriteLine($"Before ModifyValue: a = {a}");
        ModifyValue(a);
        Console.WriteLine($"After ModifyValue:  a = {a}");

        MyData data = new MyData { Value = 10 };
        Console.WriteLine($"Before ModifyReference: data.Value = {data.Value}");
        ModifyReference(data);
        Console.WriteLine($"After ModifyReference:  data.Value = {data.Value}");
    }
}
```

■ Sample Output

```
Before ModifyValue: a = 10
After ModifyValue:  a = 10
Before ModifyReference: data.Value = 10
After ModifyReference:  data.Value = 100
```

Exercise 03: Use Primary Constructors in C# 12

■ Source Code

```
using System;

class Person(string firstName, string lastName, int age)
{
    public string FirstName { get; } = firstName;
    public string LastName { get; } = lastName;
    public int Age { get; } = age;

    public void DisplayInfo()
        => Console.WriteLine($"Name: {FirstName} {LastName}, Age: {Age}");
}

class Program
{
    static void Main()
    {
        var person = new Person("Alice", "Smith", 30);
        person.DisplayInfo();
    }
}
```

■ Sample Output

Name: Alice Smith, Age: 30

Exercise 04: Demonstrate Type Inference with var and new()

■ Source Code

```
using System;

class Box { public string Label { get; set; } = "Default"; }

class Program
{
    static void Main()
    {
        var number = 42;
        var text    = "Hello";
        var box     = new Box { Label = "TypeInference" };
        Box box2    = new() { Label = "TargetTyped" };

        Console.WriteLine($"number : {number.GetType().Name} = {number}");
        Console.WriteLine($"text    : {text.GetType().Name} = {text}");
        Console.WriteLine($"box     : {box.GetType().Name} = {box.Label}");
        Console.WriteLine($"box2    : {box2.GetType().Name} = {box2.Label}");
    }
}
```

■ Sample Output

```
number : Int32 = 42
text    : String = Hello
box     : Box   = TypeInference
box2    : Box   = TargetTyped
```

Exercise 05: Perform Conditional Logic for Grade Calculation

■ Source Code

```
using System;

class Program
{
    static string GetGrade(int score) => score switch
    {
        >= 90 => "A",
        >= 80 => "B",
        >= 70 => "C",
        >= 60 => "D",
        _ => "F"
    };

    static void Main()
    {
        Console.Write("Enter your score: ");
        int score = int.Parse(Console.ReadLine() ?? "0");

        if (score >= 90) Console.WriteLine("if/else Grade: A");
        else if (score >= 80) Console.WriteLine("if/else Grade: B");
        else if (score >= 70) Console.WriteLine("if/else Grade: C");
        else if (score >= 60) Console.WriteLine("if/else Grade: D");
        else Console.WriteLine("if/else Grade: F");

        Console.WriteLine($"switch Grade : {GetGrade(score)}");
    }
}
```

■ Sample Output

```
Enter your score: 85
if/else Grade: B
switch Grade : B
```

Exercise 06: Loop Through an Array with Different Loop Types

■ Source Code

```
using System;

class Program
{
    static void Main()
    {
        int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

        Console.WriteLine("--- for loop (skip even) ---");
        for (int i = 0; i < numbers.Length; i++)
        {
            if (numbers[i] % 2 == 0) continue;
            Console.Write(numbers[i] + " ");
        }

        Console.WriteLine("\n--- foreach loop (stop at 7) ---");
        foreach (int n in numbers)
        {
            if (n == 7) break;
            Console.Write(n + " ");
        }

        Console.WriteLine("\n--- while loop ---");
        int idx = 0;
        while (idx < numbers.Length)
            Console.Write(numbers[idx++] + " ");

        Console.WriteLine("\n--- do-while loop ---");
        int j = 0;
        do { Console.Write(numbers[j++] + " "); } while (j < numbers.Length);
        Console.WriteLine();
    }
}
```

■ Sample Output

```
--- for loop (skip even) ---
1 3 5 7 9
--- foreach loop (stop at 7) ---
1 2 3 4 5 6
--- while loop ---
1 2 3 4 5 6 7 8 9 10
--- do-while loop ---
1 2 3 4 5 6 7 8 9 10
```

Exercise 07: Implement Method Overloading

■ Source Code

```
using System;

class Calculator
{
    public static int    CalculateTotal(int a, int b)
        => a + b;
    public static double CalculateTotal(double a, double b)
        => a + b;
    public static double CalculateTotal(double a, double b, double c)
        => a + b + c;
}

class Program
{
    static void Main()
    {
        Console.WriteLine($"int    + int    = {Calculator.CalculateTotal(3, 4)}");
        Console.WriteLine($"double + double = {Calculator.CalculateTotal(3.5, 4.5)}");
        Console.WriteLine($"3 doubles      = {Calculator.CalculateTotal(1.1, 2.2, 3.3)}");
    }
}
```

■ Sample Output

```
int    + int    = 7
double + double = 8
3 doubles      = 6.6
```

Exercise 08: Use ref, out, and in Parameters

■ Source Code

```
using System;

class Program
{
    static void DoubleIt(ref int x) { x *= 2; }

    static void GetMinMax(int[] arr, out int min, out int max)
    {
        min = arr[0]; max = arr[0];
        foreach (var n in arr)
        { if (n < min) min = n; if (n > max) max = n; }
    }

    static void PrintReadOnly(in int value)
        => Console.WriteLine($"in value: {value}");

    static void Main()
    {
        int val = 5;
        Console.WriteLine($"Before ref: {val}");
        DoubleIt(ref val);
        Console.WriteLine($"After  ref: {val}");

        int[] nums = { 3, 1, 7, 2, 9 };
        GetMinMax(nums, out int min, out int max);
        Console.WriteLine($"out  -> Min={min}, Max={max}");

        PrintReadOnly(in val);
    }
}
```

■ Sample Output

```
Before ref: 5
After  ref: 10
out -> Min=1, Max=9
in value: 10
```

Exercise 09: Use Local Functions

■ Source Code

```
using System;

class Program
{
    static long CalculateFactorial(int n)
    {
        long LocalFactorial(int x) =>
            x <= 1 ? 1 : x * LocalFactorial(x - 1);
        return LocalFactorial(n);
    }

    static void Main()
    {
        for (int i = 0; i <= 10; i++)
            Console.WriteLine($"{i}! = {CalculateFactorial(i)}");
    }
}
```

■ Sample Output

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Exercise 10: Demonstrate OOP Basics with Constructors

■ Source Code

```
using System;

class Car
{
    public string Make { get; }
    public string Model { get; }
    public int Year { get; }

    public Car() { Make = "Unknown"; Model = "Unknown"; Year = 2000; }
    public Car(string make, string model, int year)
    { Make = make; Model = model; Year = year; }

    public override string ToString() => $"{Year} {Make} {Model}";
}

class Program
{
    static void Main()
    {
        Car car1 = new Car();
        Car car2 = new Car("Toyota", "Camry", 2024);
        Console.WriteLine($"Car 1: {car1}");
        Console.WriteLine($"Car 2: {car2}");
    }
}
```

■ Sample Output

```
Car 1: 2000 Unknown Unknown
Car 2: 2024 Toyota Camry
```

Exercise 11: Demonstrate Access Modifiers

■ Source Code

```
using System;

class Animal
{
    public    string Name    = "Animal";
    private  string secret = "hidden";
    protected string family = "Mammal";

    public void ShowAll()
        => Console.WriteLine($"Name={Name}, Secret={secret}, Family={family}");
}

class Dog : Animal
{
    public void ShowInherited()
        => Console.WriteLine($"Dog family: {family}");
}

class Program
{
    static void Main()
    {
        Animal a = new Animal();
        a.ShowAll();
        Console.WriteLine($"Public Name: {a.Name}");

        Dog d = new Dog();
        d.ShowInherited();
    }
}
```

■ Sample Output

```
Name=Animal, Secret=hidden, Family=Mammal
Public Name: Animal
Dog family: Mammal
```

Exercise 12: Use Auto-Properties and Backing Fields

■ Source Code

```
using System;

class Product
{
    public string Name { get; set; } = "";
    private double _price;
    public double Price
    {
        get => _price;
        set => _price = value < 0
            ? throw new ArgumentException("Price cannot be negative.")
            : value;
    }
}

class Program
{
    static void Main()
    {
        var p = new Product { Name = "Laptop", Price = 999.99 };
        Console.WriteLine($"Name: {p.Name}, Price: {p.Price:C}");
        try { p.Price = -50; }
        catch (ArgumentException ex)
        { Console.WriteLine($"Validation error: {ex.Message}"); }
    }
}
```

■ Sample Output

```
Name: Laptop, Price: $999.99
Validation error: Price cannot be negative.
```

Exercise 13: Create and Use Records with init Properties

■ Source Code

```
using System;

record Employee
{
    public required string Name      { get; init; }
    public required string Department { get; init; }
    public required double Salary    { get; init; }
}

class Program
{
    static void Main()
    {
        var emp1 = new Employee
        { Name = "Alice", Department = "IT", Salary = 75000 };
        Console.WriteLine($"Original : {emp1}");

        var emp2 = emp1 with { Salary = 85000 };
        Console.WriteLine($"Modified : {emp2}");
        Console.WriteLine($"Original unchanged: {emp1}");
    }
}
```

■ Sample Output

```
Original : Employee { Name = Alice, Department = IT, Salary = 75000 }
Modified : Employee { Name = Alice, Department = IT, Salary = 85000 }
Original unchanged: Employee { Name = Alice, Department = IT, Salary = 75000 }
```

Exercise 14: Demonstrate Inheritance and Method Overriding

■ Source Code

```
using System;

class Shape
{
    public virtual void Draw() => Console.WriteLine("Drawing a Shape");
}

class Circle : Shape
{
    public override void Draw() => Console.WriteLine("Drawing a Circle");
}

class Rectangle : Shape
{
    public override void Draw() => Console.WriteLine("Drawing a Rectangle");
}

class Program
{
    static void Main()
    {
        Shape[] shapes = { new Shape(), new Circle(), new Rectangle() };
        foreach (var s in shapes) s.Draw();
    }
}
```

■ Sample Output

```
Drawing a Shape
Drawing a Circle
Drawing a Rectangle
```

Exercise 15: Differentiate Abstract Classes and Interfaces

■ Source Code

```
using System;

abstract class Vehicle { public abstract void Drive(); }

interface IDrivable { void Start(); }

class Car : Vehicle, IDrivable
{
    public override void Drive() => Console.WriteLine("Car is driving.");
    public void Start()           => Console.WriteLine("Car engine started.");
}

class Program
{
    static void Main()
    {
        Car car = new Car();
        car.Start();
        car.Drive();

        Vehicle v = new Car(); v.Drive();
        IDrivable d = new Car(); d.Start();
    }
}
```

■ Sample Output

```
Car engine started.
Car is driving.
Car is driving.
Car engine started.
```

Exercise 16: Handle Null References Safely

■ Source Code

```
using System;

class Person { public string? Name { get; set; } public Address? Address { get; set; } }
class Address { public string? City { get; set; } }

class Program
{
    static void Main()
    {
        Person? p = null;
        Console.WriteLine(p?.Name ?? "No name (person is null)");

        p = new Person { Name = "Bob", Address = null };
        Console.WriteLine(p?.Name ?? "No name");
        Console.WriteLine(p?.Address?.City ?? "No city (address is null)");

        p.Address = new Address { City = "Hyderabad" };
        Console.WriteLine(p?.Address?.City ?? "No city");
    }
}
```

■ Sample Output

```
No name (person is null)
Bob
No city (address is null)
Hyderabad
```

Exercise 17: Use Null-Conditional Chaining in a Contact App

■ Source Code

```
using System;
using System.Collections.Generic;

class Contact { public string? Name { get; set; } public string? PhoneNumber { get; set; } }

class Program
{
    static void Main()
    {
        var contacts = new List<Contact?>
        {
            new Contact { Name = "Alice", PhoneNumber = "9876543210" },
            new Contact { Name = null,      PhoneNumber = "1234567890" },
            null
        };

        foreach (var contact in contacts)
        {
            string display = contact?.Name ?? "(no name)";
            Console.WriteLine(
                $"Contact: {display} Phone: {contact?.PhoneNumber ?? "N/A"}");
        }
    }
}
```

■ Sample Output

```
Contact: Alice      Phone: 9876543210
Contact: (no name) Phone: 1234567890
Contact: (no name) Phone: N/A
```

Exercise 18: Use the required Modifier in C# 12

■ Source Code

```
using System;

class Student
{
    public required string Name { get; set; }
    public required int RollNo { get; set; }
    public string Grade { get; set; } = "N/A";
}

class Program
{
    static void Main()
    {
        var s = new Student { Name = "Ravi", RollNo = 101, Grade = "A" };
        Console.WriteLine($"Student: {s.Name}, Roll: {s.RollNo}, Grade: {s.Grade}");

        // var s2 = new Student { Name = "Priya" };
        // CS9035: Required member 'RollNo' must be set in object initializer
    }
}
```

■ Sample Output

Student: Ravi, Roll: 101, Grade: A
(Omitting RollNo causes compile error CS9035)

Exercise 19: Work with Lists and Dictionaries

■ Source Code

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        var fruits = new List<string> { "Apple", "Banana", "Cherry" };
        fruits.Add("Date");
        fruits.Remove("Banana");
        Console.WriteLine("List<string>:");
        foreach (var f in fruits) Console.WriteLine($" {f}");

        var dict = new Dictionary<int, string> {{1,"One"},{2,"Two"},{3,"Three"}};
        dict[4] = "Four";
        dict.Remove(2);
        Console.WriteLine("Dictionary<int,string>:");
        foreach (var kv in dict)
            Console.WriteLine($" {kv.Key} => {kv.Value}");
    }
}
```

■ Sample Output

```
List<string>:
Apple
Cherry
Date
Dictionary<int,string>:
1 => One
3 => Three
4 => Four
```

Exercise 20: Use LINQ for Filtering and Projection

■ Source Code

```
using System;
using System.Collections.Generic;
using System.Linq;

class Order
{
    public int    OrderId    { get; set; }
    public string CustomerName { get; set; } = "";
    public double TotalAmount { get; set; }
}

class Program
{
    static void Main()
    {
        var orders = new List<Order>
        {
            new Order { OrderId=1, CustomerName="Alice",    TotalAmount=250.0 },
            new Order { OrderId=2, CustomerName="Bob",      TotalAmount=80.0  },
            new Order { OrderId=3, CustomerName="Charlie",  TotalAmount=500.0 },
            new Order { OrderId=4, CustomerName="Diana",    TotalAmount=150.0 },
        };

        var highValue = orders
            .Where(o => o.TotalAmount > 100)
            .Select(o => new { o.OrderId, o.CustomerName, o.TotalAmount })
            .OrderByDescending(o => o.TotalAmount);

        Console.WriteLine("Orders > $100:");
        foreach (var o in highValue)
            Console.WriteLine($"  #{o.OrderId} {o.CustomerName}: ${o.TotalAmount}");
    }
}
```

■ Sample Output

```
Orders > $100:
#3 Charlie: $500
#1 Alice: $250
#4 Diana: $150
```

Exercise 21: Use Pattern Matching with is and switch

■ Source Code

```
using System;

class Program
{
    static void Describe(object obj)
    {
        if (obj is int i) Console.WriteLine($"Integer: {i}");
        else if (obj is string s) Console.WriteLine($"String of length {s.Length}: \"{s}\"");
        else if (obj is double d) Console.WriteLine($"Double: {d:F2}");
        else Console.WriteLine($"Unknown: {obj.GetType().Name}");
    }

    static string Classify(object obj) => obj switch
    {
        int n when n < 0 => $"Negative int: {n}",
        int n => $"Positive int: {n}",
        string { Length: 0 } => "Empty string",
        string s => $"Non-empty string: {s}",
        null => "null value",
        _ => $"Other: {obj}"
    };

    static void Main()
    {
        object[] items = { 42, -5, "Hello", "", 3.14, null! };
        foreach (var item in items) { Describe(item); Console.WriteLine($" => {Classify(item)}"); }
    }
}
```

■ Sample Output

```
Integer: 42
=> Positive int: 42
Integer: -5
=> Negative int: -5
String of length 5: "Hello"
=> Non-empty string: Hello
String of length 0: ""
=> Empty string
Double: 3.14
=> Other: 3.14
Unknown:
=> null value
```

Exercise 22: Create and Deconstruct Tuples

■ Source Code

```
using System;

class Program
{
    static (int Id, string Name) GetUser() => (1, "Alice");

    static void Main()
    {
        var user = GetUser();
        Console.WriteLine($"Id: {user.Id}, Name: {user.Name}");

        (int id, string name) = GetUser();
        Console.WriteLine($"Deconstructed -> Id: {id}, Name: {name}");
    }
}
```

■ Sample Output

```
Id: 1, Name: Alice
Deconstructed -> Id: 1, Name: Alice
```

Exercise 23: Simulate Async File Upload with Exception Handling

■ Source Code

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task<string> UploadFileAsync(string fileName)
    {
        Console.WriteLine($"Uploading {fileName}...");
        await Task.Delay(3000);
        if (fileName.Contains("bad"))
            throw new InvalidOperationException("Upload failed: invalid file.");
        return $"{fileName} uploaded successfully!";
    }

    static async Task Main()
    {
        foreach (var file in new[] { "report.pdf", "bad_file.exe" })
        {
            try
            {
                string result = await UploadFileAsync(file);
                Console.WriteLine(result);
            }
            catch (InvalidOperationException ex)
            {
                Console.WriteLine($"Error: {ex.Message}");
            }
        }
    }
}
```

■ Sample Output

```
Uploading report.pdf...
report.pdf uploaded successfully!
Uploading bad_file.exe...
Error: Upload failed: invalid file.
```

Exercise 24: Serialize and Deserialize JSON Files

■ Source Code

```
using System;
using System.IO;
using System.Text.Json;

class User
{
    public string Name { get; set; } = "";
    public int Age { get; set; }
    public string Email { get; set; } = "";
}

class Program
{
    static void Main()
    {
        var user = new User { Name="Alice", Age=30, Email="alice@example.com" };
        string json = JsonSerializer.Serialize(user,
            new JsonSerializerOptions { WriteIndented = true });
        Console.WriteLine("Serialized JSON:\n" + json);
        File.WriteAllText("user.json", json);

        string loaded = File.ReadAllText("user.json");
        User? deserialized = JsonSerializer.Deserialize<User>(loaded);
        Console.WriteLine(
            $"Deserialized -> Name: {deserialized?.Name}, " +
            $"Age: {deserialized?.Age}, Email: {deserialized?.Email}");
    }
}
```

■ Sample Output

```
{
  "Name": "Alice",
  "Age": 30,
  "Email": "alice@example.com"
}
Saved to user.json

Deserialized -> Name: Alice, Age: 30, Email: alice@example.com
```

Exercise 25: Use FileStream and MemoryStream

■ Source Code

```
using System;
using System.IO;
using System.Text;

class Program
{
    static void Main()
    {
        File.WriteAllText("sample.txt",
            "Hello from FileStream!\nLine 2.\nLine 3.");

        Console.WriteLine("--- FileStream read ---");
        using (var fs = new FileStream("sample.txt", FileMode.Open))
        {
            byte[] buffer = new byte[fs.Length];
            fs.Read(buffer, 0, buffer.Length);
            Console.WriteLine(Encoding.UTF8.GetString(buffer));
        }

        Console.WriteLine("--- MemoryStream write ---");
        using (var ms = new MemoryStream())
        {
            byte[] data = Encoding.UTF8.GetBytes("MemoryStream data example.");
            ms.Write(data, 0, data.Length);
            Console.WriteLine($"Bytes written: {ms.Length}");
            ms.Position = 0;
            using var reader = new StreamReader(ms);
            Console.WriteLine("Content: " + reader.ReadToEnd());
        }
    }
}
```

■ Sample Output

```
--- FileStream read ---
Hello from FileStream!
Line 2.
Line 3.
--- MemoryStream write ---
Bytes written: 26
Content: MemoryStream data example.
```

Exercise 26: Demonstrate Race Conditions with Multi-threading

■ Source Code

```
using System;
using System.Threading;

class Program
{
    static int counter = 0;
    static readonly object _lock = new object();

    static void Increment(int iterations)
    {
        for (int i = 0; i < iterations; i++)
            lock (_lock) { counter++; }
    }

    static void Main()
    {
        int n = 10000;
        var t1 = new Thread(() => Increment(n));
        var t2 = new Thread(() => Increment(n));
        var t3 = new Thread(() => Increment(n));

        t1.Start(); t2.Start(); t3.Start();
        t1.Join(); t2.Join(); t3.Join();

        Console.WriteLine($"Expected: {3 * n}, Got: {counter}");
        Console.WriteLine(counter == 3 * n
            ? "No race condition (lock worked)."
            : "Race condition occurred!");
    }
}
```

■ Sample Output

```
Expected: 30000, Got: 30000
No race condition (lock worked).
```

Exercise 27: Simulate and Resolve a Deadlock

■ Source Code

```
using System;
using System.Threading;

class Program
{
    static readonly object lockA = new object();
    static readonly object lockB = new object();

    static void Thread1()
    {
        if (Monitor.TryEnter(lockA, 500))
        {
            try {
                Console.WriteLine("T1 acquired lockA");
                Thread.Sleep(100);
                if (Monitor.TryEnter(lockB, 500))
                {
                    try { Console.WriteLine("T1 acquired lockB - done"); }
                    finally { Monitor.Exit(lockB); }
                }
                else Console.WriteLine("T1: lockB timeout (deadlock avoided)");
            }
            finally { Monitor.Exit(lockA); }
        }
    }

    static void Thread2()
    {
        if (Monitor.TryEnter(lockB, 500))
        {
            try {
                Console.WriteLine("T2 acquired lockB");
                Thread.Sleep(100);
                if (Monitor.TryEnter(lockA, 500))
                {
                    try { Console.WriteLine("T2 acquired lockA - done"); }
                    finally { Monitor.Exit(lockA); }
                }
                else Console.WriteLine("T2: lockA timeout (deadlock avoided)");
            }
            finally { Monitor.Exit(lockB); }
        }
    }

    static void Main()
    {
        var t1 = new Thread(Thread1);
        var t2 = new Thread(Thread2);
        t1.Start(); t2.Start();
        t1.Join(); t2.Join();
        Console.WriteLine("Done - no indefinite deadlock.");
    }
}
```

■ Sample Output

```
T1 acquired lockA
T2 acquired lockB
T1: lockB timeout (deadlock avoided)
T2: lockA timeout (deadlock avoided)
Done - no indefinite deadlock.
```

Exercise 28: Log with System.Diagnostics.Trace

■ Source Code

```
using System;
using System.Diagnostics;

class Logger
{
    public Logger(string logFile)
    {
        Trace.Listeners.Add(
            new TextWriterTraceListener(logFile, "fileListener"));
        Trace.Listeners.Add(new ConsoleTraceListener());
        Trace.AutoFlush = true;
    }

    public void Log(string message) =>
        Trace.WriteLine($"[{DateTime.Now:yyyy-MM-dd HH:mm:ss}] {message}");
}

class Program
{
    static void Main()
    {
        var logger = new Logger("app.log");
        logger.Log("Application started.");
        logger.Log("Processing data...");
        logger.Log("Application finished.");
        Console.WriteLine("Log written to app.log and console.");
    }
}
```

■ Sample Output

```
[2024-06-01 10:30:00] Application started.
[2024-06-01 10:30:00] Processing data...
[2024-06-01 10:30:00] Application finished.
Log written to app.log and console.
```

Exercise 29: Sanitize Input and Prevent XSS

■ Source Code

```
using System;
using System.Web;

class Program
{
    static string SanitizeInput(string input)
        => HttpUtility.HtmlEncode(input);

    static void Main()
    {
        string[] inputs = {
            "Hello, World!",
            "<script>alert('XSS Attack!')</script>",
            "<b>Bold</b> & <i>Italic</i>",
            "Normal text"
        };

        Console.WriteLine($"{"Raw Input",-45} | Sanitized Output");
        Console.WriteLine(new string('-', 90));
        foreach (var input in inputs)
            Console.WriteLine($"{{input,-45}} | {{SanitizeInput(input)}}");
    }
}
```

■ Sample Output

Raw Input	Sanitized Output
-----	-----
Hello, World!	Hello, World!
<script>alert('XSS Attack!')</script>	<script>alert('XSS Attack!')</script>
Bold & <i>Italic</i>	Bold & <i>Italic</i>
Normal text	Normal text

Exercise 30: Perform CRUD Operations using ADO.NET

■ Source Code

```
using System;
using Microsoft.Data.SqlClient;

class Program
{
    const string ConnStr =
        "Server=localhost;Database=TestDB;" +
        "Trusted_Connection=True;TrustServerCertificate=True;";

    static void Main()
    {
        using var conn = new SqlConnection(ConnStr);
        conn.Open();
        Console.WriteLine("Connected to SQL Server.");

        // CREATE TABLE
        ExecuteNonQuery(conn, @"
            IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='Employees')
            CREATE TABLE Employees (
                Id INT IDENTITY PRIMARY KEY,
                Name NVARCHAR(100),
                Position NVARCHAR(100),
                Salary DECIMAL(10,2));

        // INSERT
        ExecuteNonQuery(conn,
            "INSERT INTO Employees VALUES ('Alice','Developer',75000)");
        ExecuteNonQuery(conn,
            "INSERT INTO Employees VALUES ('Bob','Designer',65000)");
        Console.WriteLine("Inserted 2 employees.");

        // READ
        Console.WriteLine("\n--- All Employees ---");
        ReadAll(conn);

        // UPDATE
        ExecuteNonQuery(conn,
            "UPDATE Employees SET Salary=80000 WHERE Name='Alice'");
        Console.WriteLine("\nUpdated Alice's salary.");
        ReadAll(conn);

        // DELETE
        ExecuteNonQuery(conn,
            "DELETE FROM Employees WHERE Name='Bob'");
        Console.WriteLine("\nDeleted Bob.");
        ReadAll(conn);
    }

    static void ReadAll(SqlConnection conn)
    {
        using var cmd = new SqlCommand("SELECT * FROM Employees", conn);
        using var reader = cmd.ExecuteReader();
        while (reader.Read())
            Console.WriteLine(
                $" [{reader["Id"]} {reader["Name"]} | " +
                $"{reader["Position"]} | ${reader["Salary"]}");
    }

    static void ExecuteNonQuery(SqlConnection conn, string sql)
    {
        using var cmd = new SqlCommand(sql, conn);
        cmd.ExecuteNonQuery();
    }
}
```

■ Sample Output

Connected to SQL Server.
Inserted 2 employees.

--- All Employees ---
[1] Alice | Developer | \$75000.00
[2] Bob | Designer | \$65000.00

Updated Alice's salary.

```
[1] Alice | Developer | $80000.00  
[2] Bob   | Designer   | $65000.00
```

Deleted Bob.

```
[1] Alice | Developer | $80000.00
```